

AuraGTM

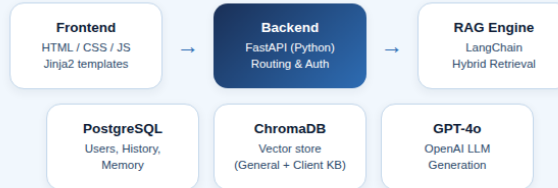
System Architecture & AI/ML Workflow

SDA Bootcamp — AI · Architecture / Workflow Submission

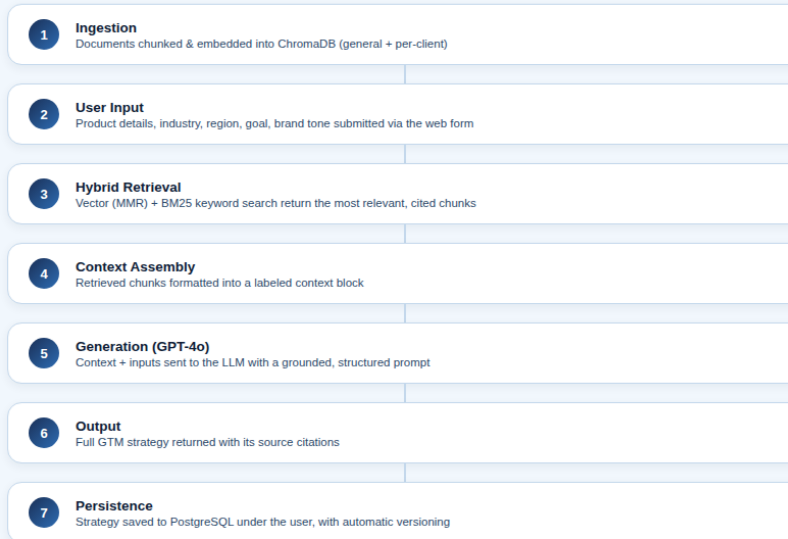
AuraGTM — System Architecture & AI/ML Workflow

SDA Bootcamp — AI / Architecture & Workflow Submission

SYSTEM ARCHITECTURE



AI / ML WORKFLOW



System Architecture

AuraGTM is built as a retrieval-augmented generation (RAG) application. The frontend sends user input to a FastAPI backend, which retrieves grounded context from a vector knowledge base and a relational database, then generates a complete go-to-market strategy using an LLM.

Component	Technology	Role
Frontend	HTML / CSS / JavaScript (Jinja2 templates)	User interface for generating strategies, viewing history, and admin tools
Backend	FastAPI (Python)	Routing, authentication, request orchestration
Authentication	Werkzeug password hashing + PostgreSQL	User signup, login, and session management
Database	PostgreSQL (SQLAlchemy ORM)	Stores users, project history, and project memory
Vector Store	ChromaDB	Separate collections for general market knowledge and per-client documents
Embeddings	HuggingFace all-MiniLM-L6-v2	Converts text chunks into vector representations
Retrieval	Hybrid search (vector MMR + BM25)	Combines semantic similarity with keyword matching
LLM	OpenAI GPT-4o (via LangChain)	Generates grounded, structured GTM strategies
Email	Gmail SMTP	Sends contact-form inquiries

AI / ML Workflow

The end-to-end pipeline, from raw documents to a saved, cited strategy:

1. Ingestion

PDF, DOCX, and TXT documents are loaded, split into chunks, and embedded into ChromaDB — separate collections for general market knowledge and each individual client.

2. User Input

The user submits product details — name, description, industry, region, business goal, and brand tone — through the web form.

3. Hybrid Retrieval

The query is used to search the relevant knowledge base(s) using a hybrid method (vector similarity via MMR, plus BM25 keyword search), returning the most relevant chunks along with their source metadata (filename and page number).

4. Context Assembly

Retrieved chunks are formatted into a structured, labeled context block that the LLM can reference directly.

5. Generation (GPT-4o)

The assembled context and user inputs are sent to GPT-4o with a structured prompt instructing it to ground every recommendation in the retrieved evidence and explain its reasoning.

6. Output

A complete, sectioned go-to-market strategy is returned to the user, along with the list of sources actually used.

7. Persistence

The generated strategy is saved to PostgreSQL under the logged-in user's account, with automatic versioning (v1, v2, v3...) if the same product is generated again.

Additional AI-Powered Features

The same retrieval-and-generation pipeline also powers several auxiliary tools, all grounded in the same knowledge base:

- Competitive Gap Analysis — strengths, weaknesses, and opportunities versus the market
- Compare 3 Directions — three distinct go-to-market strategies generated side-by-side
- What-If Scenario Simulation — re-evaluates the strategy under a hypothetical condition
- 30/60/90 Day Launch Plan — a phased execution timeline
- Brand Tone Matching — scores how well a strategy fits the intended brand voice
- Ask AuraGTM — a conversational assistant for quick, grounded GTM questions